SPRING BOOT

PRIMEIROS PASSOS COM SOFTWARE LIVRE

Criando sua primeira aplicação com Spring Boot

Igor Guerreiro
Engenheiro de Software

O QUE É SOFTWARE LIVRE?

DEFINIÇÃO

> *"Software Livre é aquele que respeita a liberdade dos usuários: você pode usar, estudar, modificar e compartilhar."*

AS 4 LIBERDADES

0 **Usar** o programa como quiser, para qualquer propósito.

1 **Estudar** como o programa funciona e adaptá-lo às suas necessidades.

2 **Distribuir** cópias para ajudar outras pessoas.

3 **Modificar** e distribuir suas versões modificadas para a comunidade.

POR QUE IMPORTA?

Gratuito para aprender e usar

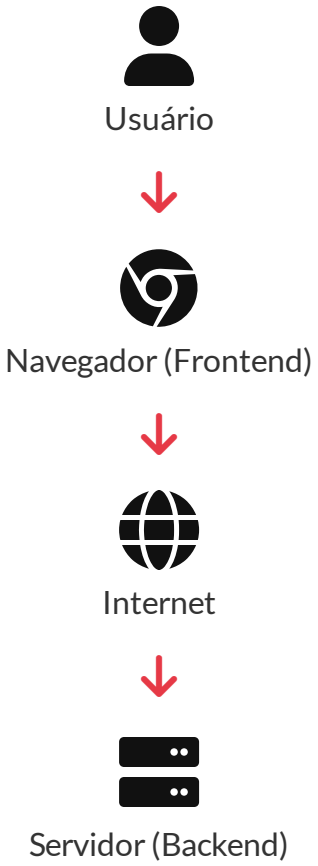
Comunidade global colaborativa

Mercado adota amplamente

Independência tecnológica

COMO FUNCIONA UMA APLICAÇÃO WEB?

O FLUXO



A DIFERENÇA FUNDAMENTAL

FRONTEND	BACKEND
O que você **vê**	A lógica por **trás**
Botões, menus, cores, layout	Cálculos, regras de negócio, dados
HTML, CSS, JavaScript	Java, Python, Node.js, Ruby
Roda no navegador do usuário	Roda no servidor da empresa

O QUE É UMA API?

DEFINIÇÃO

> *"API é um 'contrato' que define como dois sistemas se comunicam."*

A ANALOGIA DO RESTAURANTE

Você (Cliente)
Representa o **Frontend** (Navegador)

Garçom
Representa a **API**

Cozinha
Representa o **Backend** (Servidor)

> O garçom leva seu pedido à cozinha e traz o resultado. Você não precisa entrar na cozinha!

HTTP: A LÍNGUA DA WEB

Para que o Frontend e o Backend se entendam, eles usam o protocolo HTTP.

MÉTODO	PARA QUE SERVE
GET	Buscar informações (ex: listar tarefas)
POST	Enviar / criar dados (ex: nova tarefa)

O QUE É O SPRING BOOT?

DEFINIÇÃO

> *"Spring Boot é um framework open source para criar aplicações Java de forma rápida e simples."*

O QUE ELE FAZ POR VOCÊ

SEM SPRING BOOT	COM SPRING BOOT
✕ Configurar servidor manualmente	✓ Servidor já incluso e pronto
✕ Gerenciar rotas na mão	✓ Rotas com uma anotação
✕ Muito código repetitivo	✓ Código limpo e direto
✕ Configurações complexas	✓ Convenção sobre configuração

POR QUE USAR?

- **Gratuito**
- **Open Source**
- **Mercado** (Amplamente usado)
- **Comunidade** (Enorme suporte)

COMO SE ORGANIZA UMA APLICAÇÃO SPRING BOOT?

O FLUXO

APPLICATION

↓

CONTROLLER

↓

ENDPOINTS

1. CLASSE PRINCIPAL

O ponto de partida da aplicação. É aqui que o Spring Boot inicializa o servidor e configura tudo automaticamente.

```
@SpringBootApplication
public class TodolistApplication {
   public static void main(String[] args) {

SpringApplication.run(TodolistApplication.class,
args);
   }
}
```

2. CONTROLLER

Onde criamos os endpoints. Cada método dentro do Controller será responsável por responder a uma rota específica da API.

```
@RestController
public class TaskController {
// // Aqui ficam as rotas da API // Ex:
GET /tasks, POST /tasks
}
```

O QUE VAMOS CONSTRUIR?

O PROJETO

API de Lista de Tarefas

Uma aplicação backend completa para gerenciar um **To-Do List**.

FUNCIONALIDADES

- **Listar** todas as tarefas
- **Criar** uma nova tarefa

ROTAS DA API

MÉTODO	ROTA	O QUE FAZ
GET	/tasks	Retorna todas as tarefas
POST	/tasks	Adiciona uma nova tarefa

VISUALIZAÇÃO

GET /tasks

POST /tasks

AS ANOTAÇÕES ESSENCIAIS DO SPRING BOOT

A MÁGICA

No Spring Boot, usamos **anotações** (palavras que começam com @) para dar instruções ao framework sem precisar escrever configurações complexas.

@RESTCONTROLLER

```
@RestController
@RequestMapping("/tasks")
public class TaskController {
// // Aqui ficam as rotas da API //
Ex: GET /tasks, POST /tasks
}
```

↓

Diz ao Spring: "Esta classe é um controlador de API. Ela vai receber requisições da internet e devolver respostas (geralmente em formato JSON)."

@GETMAPPING

```
@GetMapping
public List<Task> list() {
   return tasks;
}
```

↓

Diz ao Spring: "Quando alguém acessar o endereço **GET /tasks** no navegador ou aplicativo, execute este método e devolva o resultado."

NOSSA API DE TAREFAS EM CÓDIGO

```java
@RestController
@RequestMapping("/tasks")
public class TodoController {

    private List<Task> tasks = new ArrayList<>();
    private Long counter = 1L;

    @PostMapping
    public Task create(@RequestBody Task task) {
        task.setId(counter++);
        task.setDone(false);
        tasks.add(task);
        return task;
    }

    @GetMapping
    public List<Task> list() {
        return tasks;
    }

}
```

DESTAQUES

`List<String>`

Guarda os dados na **memória** enquanto o servidor está rodando.

`@RequestBody`

Captura o dado que o usuário enviou no "corpo" da requisição POST.

`Conversão Automática`

O Spring Boot converte a lista do Java automaticamente para o formato **JSON** ao responder o navegador.

HORA DE TESTAR NOSSA API!

OPÇÃO 1: NAVEGADOR

A forma mais simples de testar. Basta abrir o seu navegador favorito e digitar o endereço da API.

SOMENTE GET

```
http://localhost:8080/tasks
```

* O navegador, por padrão, faz apenas requisições do tipo GET quando você digita uma URL.

OPÇÃO 2: BRUNO

Acesse o **usebruno.com** ou instale a extensão do **Bruno** no seu VS Code. Ele é uma ferramenta profissional, **100% open source** e focada em arquivos locais para testar suas APIs.

TESTANDO POST

Envie um JSON no corpo (Body) da requisição:

```
{ "title": "Estudar Spring
Boot" }
```

O RESULTADO

Após criar algumas tarefas com o POST, você pode listá-las.

TESTANDO GET

Resposta esperada ao acessar /tasks:

```
[ "Estudar Spring Boot", "Criar
minha primeira API" ]
```

> O Spring Boot converte automaticamente a sua lista do Java para o formato JSON!

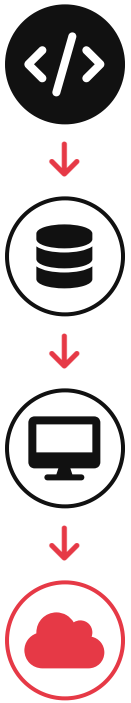
PARA ONDE IR A PARTIR DAQUI?

SUA JORNADA CONTINUA

PRÓXIMO PASSO	O QUE VOCÊ VAI APRENDER
Banco de Dados	Persistir dados de forma definitiva usando MySQL ou PostgreSQL.
JPA / Hibernate	Conectar sua aplicação Spring Boot ao banco de dados facilmente.
Frontend	Criar uma interface visual bonita com Angular, React ou Vue.js.
Deploy	Publicar sua aplicação na internet para que qualquer pessoa possa acessar.

O FUTURO

> *"Cada passo que você der hoje é a base sólida para o próximo."*



PARABÉNS! VOCÊ DEU O PRIMEIRO PASSO.

O QUE VOCÊ CONQUISTOU HOJE

- Entendeu o que é **Software Livre** e suas liberdades fundamentais.
- Compreendeu o fluxo completo: **Frontend** → **API** → **Backend**.
- Aprendeu o que é uma **API** e como o protocolo HTTP funciona.
- Criou uma **API real com Spring Boot** do absoluto zero.
- Usou ferramentas **100% gratuitas e open source** para construir tecnologia.

> ***"Todo grande sistema começou com um pequeno endpoint. Você deu o primeiro passo. Continue!"***

PERGUNTAS?

LinkedIn - Igor Guerreiro